PROJECT REPORT

Evolutionary Algorithms for Exam Timetabling

A prototype that generates semester-end examination schedules and compares GA, ES, and EP.

Problem Multi-branch exam scheduling	**Search methods** GA · ES · EP
Demo data 1,800 synthetic students	**Output** Calendar timetable + comparison dashboard

Data note: Course names, enrollment links, and USNs are fabricated for demonstration. The results are not an official college timetable.

1. Problem statement

The project assigns each exam paper to a date and session, then allocates enough halls for its enrolled students. The schedule must avoid simultaneous exams for the same student and fit within the available rooms. Among feasible schedules, the optimizer aims to reduce back-to-back papers and distribute enrollment more evenly.

Hard constraints

- A student cannot take two papers in the same session.
- Total hall demand in a session cannot exceed 40 rooms.
- A room is modeled with 60 seats; large exams can use multiple rooms.

Soft objectives

- Minimize student exam pairs in consecutive sessions.
- Reduce variation in total enrolled exam seats across sessions.

2. Data and assumptions

Branch	Code	Synthetic students
Core CSE	CS	600
AI & ML	AM	240
AI & Data Science	AD	240
CS & Business Systems	CB	240
Information Science	IS	240
CS & IoT	CI	240

The demonstration generates 73 exam entries for Semesters III–VIII. Students receive synthetic identifiers such as 1BM23CS001. The example calendar uses weekdays only, with two periods each day: 08:00–09:30 and 12:00–13:30. Hall labels are 201–220 and 401–420.

Modeling caveat: The current synthetic enrollment rule assumes students take all listed subjects for their branch and semester. Electives and actual cross-branch registrations require real enrollment data.

3. Solution workflow

1	Create synthetic students and courses
2	Build a conflict graph from shared student enrollments
3	Encode one session assignment per exam
4	Repair student conflicts and room overloads
5	Score candidate schedules
6	Compare GA, ES, and EP
7	Export CSV/JSON and display dashboard

4. Fitness function

Penalty = 1,000,000 × clashes + 1,000,000 × hall overflow + 100 × back-to-back pairs + load imbalance

Metric	Definition	Weight / target
Student clashes	Conflicting exam pairs for the same student in the same session	1,000,000; target 0
Hall overflow	Number of halls needed beyond the 40 available in a session	1,000,000; target 0
Back-to-back pairs	Student exam pairs scheduled in consecutive session numbers	100; minimize
Load imbalance	Variance-like penalty across session enrollment totals	1; minimize

Winner rule: The program ranks feasibility first, then back-to-back pairs, then load imbalance. This lexicographic ranking can select a different algorithm than sorting only by the weighted total penalty.

5. Algorithms compared

Method	Candidate variation	Selection
Genetic Algorithm (GA)	Uniform crossover between two parent schedules; mutation moves one exam to a new slot	Tournament parent choice; elites retained
Evolution Strategy (ES)	Mutation of parent schedules	(μ + λ) survivor selection
Evolutionary Programming (EP)	Mutation of each parent schedule	Pairwise competition among parents and offspring

These are educational variants. Evolutionary algorithms are stochastic search methods; there is no universally best method for every timetable. Results depend on the representation, operators, constraints, data, seed, and runtime budget.

6. Demonstration results

Algorithm	Score	Clashes	Hall over.	Back-to-back	Load imbalance	Time
GA	139,115.0	0	0	980	41,115.0	0.065s
ES	135,715.0	0	0	980	37,715.0	0.067s
EP	122,275.0	0	0	640	58,275.0	0.141s

Observed winner: EP by feasibility-first ranking, with 640 back-to-back pairs. All three methods had zero hard conflicts and zero hall overflow in this run.

Run configuration: 1,800 synthetic students; 73 exam entries; 60 sessions; population 6; 2 generations; seed 23; dates 2026-11-06 to 2026-12-17.

7. Technology stack and modules

Component	Technology	Responsibility
Optimization engine	Python 3 standard library	Dataset, conflict graph, GA/ES/EP operators, repair, scoring, room assignment
Local API	Python http.server	Serves the page and runs the optimizer from dashboard inputs
Front end	HTML, CSS, JavaScript	Neumorphic UI, calendar range, semester tables, controls, result cards
Charts	HTML Canvas API	Fitness-component and session-load charts
Data output	CSV and JSON	Timetable rows, algorithm metrics, run settings and dates

Project files: main.py (generator and algorithms), app.py (local web server), dashboard.html (user interface), output/timetable.csv, output/algorithm_comparison.csv, and output/run_summary.json.

8. Limitations and next steps

- Replace fabricated enrollment and course lists with verified registrar data.
- Model holidays, room availability by period, exam duration, accommodations, and invigilator availability.
- Improve elective enrollment representation and add validation against actual student registrations.
- Compare algorithms across repeated seeds and equal compute budgets; report averages and spread.
- Calibrate penalty weights with academic scheduling stakeholders and manually validate the final timetable.

Conclusion: The prototype demonstrates how evolutionary search can generate and compare feasible exam timetables. Its current output is a teaching/demo artifact, not an official schedule.